

Software Engineering II

Softwarearchitektur

Mathis Neunzig
ABAP Developer at SAP
mathis.neunzig@gmail.com



Disclaimer

- Diese Präsentation sowie alle weiteren Materialien dieser Vorlesung sind kein Ersatz für Literatur und können nicht als Quellen für wissenschaftliche Arbeiten genutzt werden.
- Diese Präsentation sowie alle weiteren Materialien dieser Vorlesungen dürfen nur von den Studierenden des Kurses WI 2023 SE B zur Nachbereitung der Vorlesung genutzt werden. Eine kommerzielle Nutzung und Weitergabe ist verboten.
- Alle Präsentation sowie alle weiteren Materialien werden vor Beginn der jeweiligen Vorlesung hochgeladen.
- Zur besseren Lesbarkeit wird auf die gleichzeitige Nutzung von weiblichen & männlichen Begriffen verzichtet. Es wird das generische Maskulinum verwendet. Dabei sind weibliche und sonstige Formen des Wortes gleichwertig mitgemeint. (Beispiel: statt "Kund:innen" → „Kunden“)

Softwarearchitektur

Was ist eine Softwarearchitektur?

Architektur

- Der Alltag ist immer stärker durchdrungen von Software
- Software bestimmt immer mehr den Geschäftserfolg
- Die Größe von Systemen wächst kontinuierlich
- Unsere Fähigkeit, Softwaresysteme zu entwickeln, wächst langsamer als der Bedarf (siehe Softwarekrise aus dem 3. Semester)

Architektur

- Softwareentwicklung wird aufgefasst als Ingenieursdisziplin mit

- Methoden
- Techniken
- Werkzeugen

die teilweise von traditionellen Disziplinen abgeschaut werden

- Komplexität wird begegnet mit dem Handwerkszeug des Ingenieurs

- Teile und Herrsche
- Abstraktion
- Wiederverwendung

➔ Softwarearchitektur ist ein Teil des Softwareengineerings!
(mit vielen Analogien zur klassischen Architektur)

Olympiastadion München

- Gewinner des Architekturwettbewerb:
G. Behnisch
- Gebaut 1966-1972
- Zeltdach mit 58 Pylonen
- Denkmalschutz seit 1977



Olympiastadion München

- Anforderungen
 - Langlebigkeit
 - Vielfältige Einsatzmöglichkeiten nach den Olympischen Spielen
 - Dauerhaftigkeit der Strukturen
 - Geringe Wartungskosten
 - Ästhetik
 - Imagetransport
 - Berücksichtigung lokaler Gegebenheiten
 - Brachgelände
 - Angrenzender Schuttberg
 - Balance zwischen Qualität, Zeit und Kosten
 - Begrenztes Budget
 - Hoher Zeitdruck → musste bis zur Olympiade fertig sein!

Die Disziplin „Architektur“

- „Architektur ist die Kombination von utilitas, firmitas und venustas“

Die Disziplin „Architektur“

- „Architektur ist die Kombination von utilitas, firmitas und venustas“
- Utilitas: Das Gebäude erfüllt seine Funktion
- Firmitas: Das Gebäude ist stabil
- Venustas: Das Gebäude ist ästhetisch gestaltet

Die Disziplin „Architektur“

- „Architektur ist die Kombination von utilitas, firmitas und venustas“
- Utilitas: Das Gebäude erfüllt seine Funktion
 - Software erfüllt Anforderungen
- Firmitas: Das Gebäude ist stabil
 - Software ist langlebig und stabil gegenüber Änderungen und Skalierung
- Venustas: Das Gebäude ist ästhetisch gestaltet
 - Software weist klare, logische Strukturen auf, die das Verständnis vereinfachen

Softwarearchitektur

- Es gibt über 90 Definitionen des Begriffes “Softwarearchitektur“

Softwarearchitektur

- Es gibt über 90 Definitionen des Begriffes “Softwarearchitektur”
- IEEE: „Architecture is defined [...] as the fundamental organization of a system, embodied in its components, their relationships to each other and to the environment, and the principles governing its design and evolution“

Softwarearchitektur

- Es gibt über 90 Definitionen des Begriffes “Softwarearchitektur”
- IEEE 1471-2000: „Architecture is defined [...] as the fundamental organization of a system, embodied in its components, their relationships to each other and to the environment, and the principles governing its design and evolution“
- Garlan & Perry, 1995: „The structure of the components of a program/system, their relationships and principles and guidelines governing their design and evolution over time

Softwarearchitektur

- Jonson, 2003: „In most successful software projects, the expert developers working on [those projects] have a shared understanding of the system design. This shared understanding of the system is called „architecture“. This understanding includes how the system is divided into components and how the components interact through interfaces“

Softwarearchitektur

- Jonson, 2003: „In most successful software projects, the expert developers working on [those projects] have a shared understanding of the system design. This shared understanding of the system is called „architecture“. This understanding includes how the system is divided into components and how the components interact through interfaces“
- De Marco: „Architecture is a framework for change“

Softwarearchitektur

- Jonson, 2003: „In most successful software projects, the expert developers working on [those projects] have a shared understanding of the system design. This shared understanding of the system is called „architecture“. This understanding includes how the system is divided into components and how the components interact through interfaces“
- De Marco: „Architecture is a framework for change“
- ...

→ Welche nehmen wir?

Softwarearchitektur

- Definition nach ISO/IEC/IEEE 42010:
„Die Softwarearchitektur beschreibt die grundlegenden Strukturen eines Softwaresystems, bestehend aus Softwareelementen, deren Beziehungen zueinander und den Eigenschaften dieser Elemente und Beziehungen.“
- Softwarearchitektur ist nicht nur Technik, sondern auch Organisation und Prozesse
- Abgrenzung zum Software-Design und der Implementierungsphase
 - Architektur = „Big Picture“, langfristige Strukturentscheidung
 - Design = detaillierte Struktur einzelner Komponenten
 - Code = konkrete Implementierung

Softwarearchitektur

- Definition nach ISO/IEC/IEEE 42010:
„Die Softwarearchitektur beschreibt die grundlegenden Strukturen eines Softwaresystems, bestehend aus Softwareelementen, deren Beziehungen zueinander und den Eigenschaften dieser Elemente und Beziehungen.“
- Softwarearchitektur ist nicht nur Technik, sondern auch Organisation und Prozesse
- Abgrenzung zum Software-Design und der Implementierungsphase
 - Architektur = „Big Picture“, langfristige Strukturentscheidung
 - Design = detaillierte Struktur einzelner Komponenten
 - Code = konkrete Implementierung

Softwarearchitektur

- Ziele
 - Wartbarkeit
 - Erweiterbarkeit
 - Wiederverwendbarkeit
 - Skalierbarkeit
 - Testbarkeit
 - Performance
 - Sicherheit
- Frühe Entscheidungsfindung mit langfristiger Wirkung!
 1. Architektur
 2. Design
 3. Code

Negativbeispiel

- „Ich habe schon mal eine schlechte Softwarearchitektur gesehen, weil sie...“
 - Kaum zu verstehen war
 - Die Wiederverwendung von Bestandteilen nicht zuließ
 - Zu einer schlechten Performance des Systems führte
 - Bei Änderungen der Anforderungen Anpassungen an ganz unterschiedlichen Stellen erforderlich machte
 - Nicht erweiterbar war
 - Nicht wartbar war
 - Sehr umfangreiche Schnittstellen definierte

Positivbeispiel

- Architektur eines Compilers
- Etabliertes Verständnis der einzelnen Bestandteile und ihrer Aufgaben, Verantwortlichkeiten und Abhängigkeiten
- Nutzt Best-Practices (weil man schon viele Compiler gebaut hat)



Positivbeispiel

- Warum ist die Architektur gut?
 - Einfach und verständlich
 - Effizient realisierbar
 - Langlebig und erweiterbar
 - Skalierbar bzw. „parallelisierbar“

Softwarearchitekt

Was ist ein Softwarearchitekt?

Was macht ein Softwarearchitekt?

Softwarearchitekt

- Aufgaben
 - Entwurf der System- und Softwarearchitektur
 - Umsetzung der Anforderungen an die Komponenten
 - Verantwortlichkeit für Implementierungs-, Integrations- und Prüfkonzert

Softwarearchitekt

- Fähigkeiten

- Kennt Anwendung, Rahmenbedingungen und Einsatzgebiete des Systems
- Kennt die Schnittstellen des Systems
- Kennt Architekturprinzipien und verschiedene SW-Architekturen
- Kennt Standard-Software für System- und Softwaremodellierung
- Kennt die wichtigen Methoden und Werkzeuge
- Kann Schwachstellen und Risiken in Software erkennen

Softwarearchitekt

- Fähigkeiten
 - Kann Probleme unter adäquater Berücksichtigung der Software und Hardware analysieren und Lösungsvorschläge ausarbeiten
 - Kann Probleme abstrahieren und vereinfachen
 - Kann Probleme einfach erklären
 - Kann Abhängigkeiten von Systemen erkennen und analysieren
 - Kann mit einem Team arbeiten und mit Teammitgliedern kommunizieren

Softwarearchitekt



Architekturmuster

- Architekturmuster sind Lösungen für wiederkehrende Probleme
- Es gibt keine einheitliche Einteilung der Architekturmuster
- Man kann (und soll) mehrere Muster kombinieren
 - Ähnlich wie bei Designmustern
 - z.B. die Factory als Singleton bereitstellen
 - z.B. ein Builder, der eine Strategy baut
- Jedes Muster bringt Trade-offs mit sich

Architekturmuster

- Strukturmuster
Wie ist das System intern strukturiert?
- Verteilungsmuster
Wie verteilt sich das System über mehrere Knoten?
- Verantwortlichkeitstrennungsmuster
Wie wird Komplexität durch Zuständigkeiten reduziert?
- Deployment- & Betriebsmodelle
Wie wird das System betrieben und skaliert?
- Integrationsmuster
Wie arbeiten Systeme oder Module zusammen?
- Kommunikationsmuster (nicht wirklich Teil von Architekturmustern)
Wie kommunizieren Komponenten miteinander?

Architekturmuster

Architekturmuster

Strukturmuster

- Layered Architecture
- MVC / MVP / MVVM
- Pipe-and-Filter
- Microkernel (Plug-in Architecture)
- Modular Monolith
- Component-Based Architecture
- Self-contained Systems (SCS)
- Blackboard Architecture

Architekturmuster

Strukturmuster

- Layered Architecture
- MVC / MVP / MVVM
- Pipe-and-Filter
- Microkernel (Plug-in Architecture)
- Modular Monolith
- Component-Based Architecture
- Self-contained Systems (SCS)
- Blackboard Architecture

Verteilungsmuster

- Client-Server
- Microservices
- Service-Oriented Architecture (SOA)
- Peer-to-peer (P2P)
- Event-Driven Architecture (EDA)
- Space-Based Architecture
- Shared Nothing Architecture
- Service Mesh

Architekturmuster

Strukturmuster

- Layered Architecture
- MVC / MVP / MVVM
- Pipe-and-Filter
- Microkernel (Plug-in Architecture)
- Modular Monolith
- Component-Based Architecture
- Self-contained Systems (SCS)
- Blackboard Architecture

Verteilungsmuster

- Client-Server
- Microservices
- Service-Oriented Architecture (SOA)
- Peer-to-peer (P2P)
- Event-Driven Architecture (EDA)
- Space-Based Architecture
- Shared Nothing Architecture
- Service Mesh

Verantwortlichkeits-trennungsmuster

- Domain-Driven-Design (DDD)
- Command Query Responsibility Segregation (CQRS)
- Hexagonal Architecture
- Onion Architecture
- Clean Architecture
- Screaming Architecture

Architekturmuster

Strukturmuster

- Layered Architecture
- MVC / MVP / MVVM
- Pipe-and-Filter
- Microkernel (Plug-in Architecture)
- Modular Monolith
- Component-Based Architecture
- Self-contained Systems (SCS)
- Blackboard Architecture

Verteilungsmuster

- Client-Server
- Microservices
- Service-Oriented Architecture (SOA)
- Peer-to-peer (P2P)
- Event-Driven Architecture (EDA)
- Space-Based Architecture
- Shared Nothing Architecture
- Service Mesh

Verantwortlichkeits-trennungsmuster

- Domain-Driven-Design (DDD)
- Command Query Responsibility Segregation (CQRS)
- Hexagonal Architecture
- Onion Architecture
- Clean Architecture
- Screaming Architecture

Deployment- & Betriebsmodelle

- Serverless Architecture
- Containerized Architecture
- Backend for Frontend
- API Gateway Pattern
- Sidecar Pattern
- Multi-Tenant Architecture
- Twelve-Factor App
- Immutable Infrastructure

Architekturmuster

Strukturmuster

- Layered Architecture
- MVC / MVP / MVVM
- Pipe-and-Filter
- Microkernel (Plug-in Architecture)
- Modular Monolith
- Component-Based Architecture
- Self-contained Systems (SCS)
- Blackboard Architecture

Verteilungsmuster

- Client-Server
- Microservices
- Service-Oriented Architecture (SOA)
- Peer-to-peer (P2P)
- Event-Driven Architecture (EDA)
- Space-Based Architecture
- Shared Nothing Architecture
- Service Mesh

Verantwortlichkeits-trennungsmuster

- Domain-Driven-Design (DDD)
- Command Query Responsibility Segregation (CQRS)
- Hexagonal Architecture
- Onion Architecture
- Clean Architecture
- Screaming Architecture

Deployment- & Betriebsmodelle

- Serverless Architecture
- Containerized Architecture
- Backend for Frontend
- API Gateway Pattern
- Sidecar Pattern
- Multi-Tenant Architecture
- Twelve-Factor App
- Immutable Infrastructure

Integrationsmuster

- Adapter
- Facade
- Message Bus & Message Queue
- Publish-Subscribe
- Saga Pattern
- Anti-Corruption Layer
- Gateway
- Event-Carried State Transfer
- Shared Database

Architekturmuster

Strukturmuster	Verteilungsmuster	Verantwortlichkeits-trennungsmuster	Deployment- & Betriebsmodelle	Integrationsmuster
▪ Layered Architecture ▪ MVC / MVP / MVVM ▪ Pipe-and-Filter ▪ Microkernel (Plug-in Architecture) ▪ Modular Monolith ▪ Component-Based Architecture ▪ Self-contained Systems (SCS) ▪ Blackboard Architecture	▪ Client-Server ▪ Microservices ▪ Service-Oriented Architecture (SOA) ▪ Peer-to-peer (P2P) ▪ Event-Driven Architecture (EDA) ▪ Space-Based Architecture ▪ Shared Nothing Architecture ▪ Service Mesh	▪ Domain-Driven-Design (DDD) ▪ Command Query Responsibility Segregation (CQRS) ▪ Hexagonal Architecture ▪ Onion Architecture ▪ Clean Architecture ▪ Screaming Architecture	▪ Serverless Architecture ▪ Containerized Architecture ▪ Backend for Frontend ▪ API Gateway Pattern ▪ Sidecar Pattern ▪ Multi-Tenant Architecture ▪ Twelve-Factor App ▪ Immutable Infrastructure	▪ Adapter ▪ Facade ▪ Message Bus & Message Queue ▪ Publish-Subscribe ▪ Saga Pattern ▪ Anti-Corruption Layer ▪ Gateway ▪ Event-Carried State Transfer ▪ Shared Database

Kommunikationsmuster (nicht offiziell Teil von Architekturmustern): REST, GraphQL, SOAP, Odata, Websocket, Remote Procedure Call, Graph-based API, JSON-RPC, XML-RPC

Architekturmuster

Was wir in der VL machen
ist violett markiert!

Strukturmuster

- Layered Architecture
- MVC / MVP / MVVM
- Pipe-and-Filter
- Microkernel (Plug-in Architecture)
- Modular Monolith
- Component-Based Architecture
- Self-contained Systems (SCS)
- Blackboard Architecture

Verteilungsmuster

- Client-Server
- Microservices
- Service-Oriented Architecture (SOA)
- Peer-to-peer (P2P)
- Event-Driven Architecture (EDA)
- Space-Based Architecture
- Shared Nothing Architecture
- Service Mesh

Verantwortlichkeits-trennungsmuster

- Domain-Driven-Design (DDD)
- Command Query Responsibility Segregation (CQRS)
- Hexagonal Architecture
- Onion Architecture
- Clean Architecture
- Screaming Architecture

Deployment- & Betriebsmodelle

- Serverless Architecture
- Containerized Architecture
- Backend for Frontend
- API Gateway Pattern
- Sidecar Pattern
- Multi-Tenant Architecture
- Twelve-Factor App
- Immutable Infrastructure

Integrationsmuster

- Adapter
- Facade
- Message Bus & Message Queue
- Publish-Subscribe
- Saga Pattern
- Anti-Corruption Layer
- Gateway
- Event-Carried State Transfer
- Shared Database

Kommunikationsmuster (nicht offiziell Teil von Architekturmustern): REST, GraphQL, SOAP, Odata, Websocket, Remote Procedure Call, Graph-based API, JSON-RPC, XML-RPC

Danke!

Mathis Neunzig
mathis.neunzig@gmail.com

